Real-Time Communication: WebSocket, Socket.io, Chat Implementation

This covers real-time patterns, WebSocket basics, Socket.io, and chat application design.

1) WebSocket Basics

1.1 WebSocket Protocol

WebSocket is a bidirectional, persistent connection over TCP.

Advantages over polling:

- full-duplex: both client and server can send anytime.
- lower latency: no request-response cycle.
- lower bandwidth: no HTTP headers on every message.

```
// Browser
const ws = new WebSocket("ws://localhost:8080");

ws.addEventListener("open", () => {
  console.log("connected");
  ws.send(JSON.stringify({ type: "HELLO" }));
});

ws.addEventListener("message", (event) => {
  const data = JSON.parse(event.data);
  console.log("received:", data);
});

ws.addEventListener("close", () => {
  console.log("disconnected");
});

ws.addEventListener("error", (error) => {
  console.error("websocket error:", error);
});

ws.close();
```

1.2 Node.js WebSocket Server

```
const WebSocket = require("ws");
const http = require("http");

const server = http.createServer();
const wss = new WebSocket.Server({ server });

wss.on("connection", (ws) => {
```

```
  console.log("client connected");

  ws.on("message", (message) => {
    console.log("received:", message);

    // echo to all clients
    wss.clients.forEach((client) => {
      if (client.readyState === WebSocket.OPEN) {
        client.send(message);
      }
    });
  });

  ws.on("close", () => {
    console.log("client disconnected");
  });

  ws.on("error", (error) => {
    console.error("error:", error);
  });
});

server.listen(8080, () => {
  console.log("WebSocket server listening on port 8080");
});
```

1.3 Connection States

```
console.log(ws.readyState);
// WebSocket.CONNECTING (0)
// WebSocket.OPEN (1)
// WebSocket.CLOSING (2)
// WebSocket.CLOSED (3)
```

2) Socket.io: Real-Time Engine

Socket.io provides:

- automatic reconnection.
- fallback to polling if WebSocket unavailable.
- rooms and namespaces.
- ack callbacks.

2.1 Socket.io Basic Setup

Client:

```
import { io } from "socket.io-client";

const socket = io("http://localhost:3000", {
  reconnection: true,
```

```
  reconnectionDelay: 1000,
  reconnectionDelayMax: 5000,
  reconnectionAttempts: 5
});

socket.on("connect", () => {
  console.log("connected with id:", socket.id);
});

socket.emit("message", { text: "hello" });

socket.on("message", (data) => {
  console.log("received:", data);
});

socket.on("disconnect", () => {
  console.log("disconnected");
});
```

Server:

```
const express = require("express");
const { createServer } = require("http");
const { Server } = require("socket.io");

const app = express();
const server = createServer(app);
const io = new Server(server);

io.on("connection", (socket) => {
  console.log("user connected:", socket.id);

  socket.on("message", (data, callback) => {
    console.log("received:", data);

    // send to all
    io.emit("message", {
      from: socket.id,
      text: data.text
    });

    // ack callback
    callback({ status: "received" });
  });

  socket.on("disconnect", () => {
    console.log("user disconnected:", socket.id);
  });
});

server.listen(3000, () => {
```

```
  console.log("server listening on port 3000");
});
```

2.2 Socket.io Rooms

```
// Server: join user to room
io.on("connection", (socket) => {
  socket.on("join_room", (roomId) => {
    socket.join(roomId);
    socket.to(roomId).emit("user_joined", { id: socket.id });
  });

  socket.on("message", (roomId, message) => {
    io.to(roomId).emit("message", {
      from: socket.id,
      text: message
    });
  });

  socket.on("leave_room", (roomId) => {
    socket.leave(roomId);
    socket.to(roomId).emit("user_left", { id: socket.id });
  });
});

// Client: join room
socket.emit("join_room", "room123");

// Listen for users in room
socket.on("user_joined", (user) => {
  console.log("user joined:", user.id);
});

socket.on("user_left", (user) => {
  console.log("user left:", user.id);
});

// Send message to room
socket.emit("message", "room123", "hello everyone");

// Listen to messages
socket.on("message", (data) => {
  console.log(`${data.from}: ${data.text}`);
});
```

2.3 Socket.io Namespaces

```
// Server
const chat = io.of("/chat");
```

```
const notifications = io.of("/notifications");

chat.on("connection", (socket) => {
  console.log("chat user connected");
  socket.on("message", (data) => {
    chat.emit("message", data);
  });
});

notifications.on("connection", (socket) => {
  console.log("notification subscriber connected");
  socket.on("subscribe", (userId) => {
    socket.join(`user_${userId}`);
  });
});

// Notify specific user
notifications.to(`user_123`).emit("notification", {
  type: "new_message",
  text: "You have a new message"
});

// Client
const chatSocket = io("http://localhost:3000/chat");
const notifSocket = io("http://localhost:3000/notifications");

chatSocket.emit("message", { text: "hello" });
notifSocket.emit("subscribe", "user123");

notifSocket.on("notification", (data) => {
  console.log("notification:", data.text);
});
```

3) Chat Application Architecture

3.1 React Chat Component

```
import React, { useState, useEffect, useRef } from "react";
import { io } from "socket.io-client";

function ChatRoom({ roomId, userName }) {
  const [messages, setMessages] = useState([]);
  const [input, setInput] = useState("");
  const [users, setUsers] = useState([]);
  const socketRef = useRef(null);
  const messagesEndRef = useRef(null);

  useEffect(() => {
    socketRef.current = io("http://localhost:3000");
```

```
    socketRef.current.on("connect", () => {
      socketRef.current.emit("join_room", {
        roomId,
        userName
      });
    });

    socketRef.current.on("user_joined", (user) => {
      setUsers(prev => [...prev, user]);
      setMessages(prev => [...prev, {
        type: "system",
        text: `${user.userName} joined`
      }]);
    });

    socketRef.current.on("message", (message) => {
      setMessages(prev => [...prev, message]);
    });

    socketRef.current.on("user_left", (user) => {
      setUsers(prev => prev.filter(u => u.id !== user.id));
      setMessages(prev => [...prev, {
        type: "system",
        text: `${user.userName} left`
      }]);
    });

    return () => {
      socketRef.current.emit("leave_room", roomId);
      socketRef.current.disconnect();
    };
  }, [roomId, userName]);

  useEffect(() => {
    // auto-scroll to bottom
    messagesEndRef.current?.scrollIntoView({ behavior: "smooth" });
  }, [messages]);

  const sendMessage = (e) => {
    e.preventDefault();
    if (input.trim()) {
      socketRef.current.emit("send_message", {
        roomId,
        text: input,
        userName
      });
      setInput("");
    }
  };

  return (
    <div className="chat">
```

```
      <div className="users">
        <h3>Users ({users.length})</h3>
        {users.map(u => (
          <div key={u.id} className="user">{u.userName}</div>
        ))}
      </div>

      <div className="messages">
        {messages.map((msg, i) => (
          <div key={i} className={`message ${msg.type}`}>
            {msg.type === "system" ? (
              <em>{msg.text}</em>
            ) : (
              <>
                <strong>{msg.userName}:</strong>
                <p>{msg.text}</p>
              </>
            )}
          </div>
        ))}
        <div ref={messagesEndRef} />
      </div>

      <form onSubmit={sendMessage}>
        <input
          value={input}
          onChange={(e) => setInput(e.target.value)}
          placeholder="Type message..."
        />
        <button type="submit">Send</button>
      </form>
    </div>
  );
}

export default ChatRoom;
```

3.2 Server-Side Chat Logic

```
const express = require("express");
const { createServer } = require("http");
const { Server } = require("socket.io");
const cors = require("cors");

const app = express();
app.use(cors());
const server = createServer(app);
const io = new Server(server, {
  cors: { origin: "*" }
});
```

```
const rooms = {}; // { roomId: { users, messages } }

io.on("connection", (socket) => {
  socket.on("join_room", (data) => {
    const { roomId, userName } = data;

    if (!rooms[roomId]) {
      rooms[roomId] = { users: [], messages: [] };
    }

    const user = { id: socket.id, userName };
    rooms[roomId].users.push(user);

    socket.join(roomId);

    // notify others
    socket.to(roomId).emit("user_joined", user);

    // send user list and history
    socket.emit("init", {
      users: rooms[roomId].users,
      messages: rooms[roomId].messages
    });
  });

  socket.on("send_message", (data) => {
    const { roomId, text, userName } = data;

    const message = {
      id: socket.id,
      userName,
      text,
      timestamp: Date.now()
    };

    rooms[roomId].messages.push(message);

    // keep only last 100 messages
    if (rooms[roomId].messages.length > 100) {
      rooms[roomId].messages.shift();
    }

    io.to(roomId).emit("message", message);
  });

  socket.on("leave_room", (roomId) => {
    const user = rooms[roomId]?.users.find(u => u.id === socket.id);
    if (user) {
      rooms[roomId].users = rooms[roomId].users.filter(u => u.id !== socket.id);
      io.to(roomId).emit("user_left", user);
    }
    socket.leave(roomId);
```

```
  });

  socket.on("disconnect", () => {
    // clean up
    Object.keys(rooms).forEach(roomId => {
      const user = rooms[roomId]?.users.find(u => u.id === socket.id);
      if (user) {
        rooms[roomId].users = rooms[roomId].users.filter(u => u.id !== socket.id);
        io.to(roomId).emit("user_left", user);
      }
    });
  });
});

server.listen(3000, () => {
  console.log("server listening on port 3000");
});
```

4) Handling Connection Issues

4.1 Reconnection Strategy

```
const socket = io("http://localhost:3000", {
  reconnection: true,
  reconnectionDelay: 1000, // start with 1s
  reconnectionDelayMax: 5000, // max 5s
  reconnectionAttempts: Infinity,
  randomizationFactor: 0.1
});

socket.on("disconnect", (reason) => {
  if (reason === "io server disconnect") {
    // server explicitly disconnected, reconnect manually
    socket.connect();
  }
});

socket.on("connect_error", (error) => {
  if (error.response?.status === 401) {
    // authentication issue
    localStorage.removeItem("token");
    window.location.href = "/login";
  }
});
```

4.2 Message Queue During Disconnection

```
class ChatClient {
  constructor(socket) {
```

```
    this.socket = socket;
    this.messageQueue = [];
    this.isConnected = false;

    this.socket.on("connect", () => {
      this.isConnected = true;
      this.flushMessageQueue();
    });

    this.socket.on("disconnect", () => {
      this.isConnected = false;
    });
  }

  sendMessage(message) {
    if (this.isConnected) {
      this.socket.emit("message", message);
    } else {
      // queue for later
      this.messageQueue.push(message);
    }
  }

  flushMessageQueue() {
    while (this.messageQueue.length > 0) {
      const message = this.messageQueue.shift();
      this.socket.emit("message", message);
    }
  }
}
```

5) Revision Checklist

- [ ] Understand WebSocket protocol.
- [ ] Know Socket.io features and differences.
- [ ] Implement rooms and namespaces.
- [ ] Implement chat application.
- [ ] Handle reconnection and connection issues.
- [ ] Queue messages during disconnection.
- [ ] Handle user join/leave events.